UNIT 14 QUALITY MANAGEMENT

NOTES

Structure

14.0 Introduction
14.1 Objectives
14.2 Quality Concepts
14.3 Software Quality Assurance Group
14.4 Software Quality Assurance Activities
14.5 Software Reviews
14.6 Evaluation of Quality
14.7 Software Reliability
14.8 Answers to Check Your Progress Questions
14.9 Summary
14.10 Key Words
14.11 Self Assessment Questions and Exercises
14.12 Further Readings

14.0 INTRODUCTION

Nowadays, quality has become an important factor to be considered while developing software. This is due to the fact that users are interested in quality software, which is according to their requirements and is delivered within a specified time. Furthermore, users require software, which is maintainable and minimizes the time and cost for correcting problems in it. The objective of the software development team is to design the software with minimum errors and required functionality according to user specified requirements. Software quality depends on various characteristics such as correctness, reliability, and efficiency. These characteristics are used as a checklist to implement quality in software.

14.1 OBJECTIVES

After going through this unit, you will be able to:

- Define software quality and software quality assurance
- Understand software reviews that are a systematic evaluation of the software in order to detect errors
- Explain formal technical review
- Discuss the evaluation of software quality
- Explain the process of software reliability

NOTES

14.2 QUALITY CONCEPTS

Quality refers to the features and characteristics of a product or service, which define its ability to satisfy user requirements. Generally, some concepts are used to develop high-quality software. These concepts give an overview of characteristics of the software in order to accomplish user requirements in it. Various quality concepts are listed below.

- Conformity with established standards and end user requirements
- Ease of use and operation
- Ease of modification of existing functions and addition of new functions
- Performance as expected by the user
- Well tested for functionality, user interface, and performance
- Well documented.

Software quality refers to the extent to which the software is able to perform correctly along with required features and functions. For this, a planned and systematic set of activities is performed. **IEEE** defines software quality as '*(1) the degree to which a system, component, or process meets specified requirements. (2) the degree to which a system, component, or process meets customer or user needs or expectations.*'

Principles followed for achieving good quality software are listed below.

- **Prevention of errors:** In order to avoid errors in the code, software tools are extensively used.
- **Detection and correction of errors:** Errors should be detected in the early stages of software development, so that much time is not consumed to correct them in the later stages. Thus, quality control should be carried out during all the phases of the SDLC.
- **Elimination of cause(s) of error:** After detection of errors, it is necessary to eliminate their cause. Eliminating causes of errors increases the quality of the software.
- **Independent audit:** The objective of an audit is to assess the product and process activities. Thus, independent audit for product and process should be conducted in accordance with the standards and procedures established in the quality process.

Software Quality Control

Software quality control is concerned with *product* quality and checks, whether the product meets user requirements and is developed in accordance with the established standards and procedures. In addition, it is used to accomplish the following objectives.

- Comparison of product quality procedures with the established standards
- Identification of errors in order to rectify them
- Focus on reviews and testing the product
- Identification of outputs generated by the product.

NOTES

Software Quality Assurance (SQA)

Software quality assurance is concerned with *process* quality and refers to planned and systematic sets of activities, which ensure that software life cycle processes and products conform to requirements, standards, and procedures. In addition, SQA assures that the standards, processes, and procedures are appropriate for the software project and are correctly implemented. Note that a high-quality product can be developed if there is an appropriate process quality. The objectives of SQA are listed below.

- To determine the software objectives to be accomplished
- To establish software plans, when the objectives are determined
- To monitor and adjust software plans to satisfy user requirements.

Statistical Software Quality Assurance

Statistical SQA is a technique which measures the quality using quantitative techniques. It means information about defects in collected and categorized and an attempt is made to trace defect to underlying cause.

Quality Costs

Costs incurred in performing quality-related activities are referred to as quality costs. These activities include preventing, finding, and correcting errors in the software. The quality costs are broadly divided into three categories (see Figure 14.1), which are listed below.

- **Prevention costs:** These costs are incurred while examining the activities that affect software quality during the software development. These costs are particularly incurred to prevent developing poor quality software. For example, preventive costs are used to avoid errors in coding and designing. In addition, it includes the cost incurred on correcting mistakes in user manuals, staff training, requirements analysis, and fault tolerant design of software.
- **Appraisal costs:** These costs are incurred in verifying or evaluating the product before the software is delivered to the user. For example, appraisal costs include cost on conducting code inspection, white box testing, black box testing, training testers, and beta testing.
- **Failure costs:** These costs are incurred when the product does not meet user requirements. For example, failure costs are incurred in fixing errors

NOTES

while dealing with user complaints. Failure costs are further divided into following categories (see Figure 14.1).

- **Internal failure costs:** These costs are incurred *before* the software is delivered to the user. When software is unable to perform properly due to errors in it, internal failure costs are incurred for missed milestones and the overtime done to complete these milestones within the specified schedule. For example, internal failure costs are incurred in fixing errors that are detected in regression testing.
- **External failure costs:** These costs are incurred *after* the software is delivered to the user. When errors are detected after software is delivered, a lot of time and money is consumed to fix the errors. For example, external failure costs are incurred on the modification of software, technical support calls, warranty costs, and other costs imposed by law.



Fig. 14.1 *Types of Quality Costs*

Note: The total quality cost incurred on the software is the sum of the costs shown in Figure 14.1.

ISO 9126 Quality Factors

Software is said to be of high-quality if it is free from errors and is according to user requirements. In order to obtain the desired quality, factors defined in ISO 9126 are followed. Figure 14.2 shows the factors responsible for software quality.

These factors, which act as a checklist to assess the quality of software (see Figure 14.3) are discussed here.

- **Functionality:** The capability of software to achieve the intended purpose using functions and their specified properties is referred to as functionality. The attributes of functionality are listed below.
  - **Suitability:** Ability of software to perform a set of functions that are used for specified tasks
  - **Accuracy:** Appropriateness of the results in accordance with the user requirements

NOTES

- **Interoperability:** Ability of software to interact with other systems
- **Security:** Prevention of unauthorized access to programs in software.

- **Reliability:** The capability of software to maintain its performance level under stated conditions for a period of time is referred to as reliability. The attributes of reliability are listed below.
  - **Fault tolerance:** Ability of software to perform its functions when unexpected faults occur
  - **Maturity:** Ability of software to handle frequency of failures
  - **Recoverability:** Ability of software to recover data that is lost due to software failure.
- **Usability:** The capability of software to be easily operated and understood is referred to as usability. The attributes of usability are listed below.
  - **Understandability:** The ease with which software can be understood by the user
  - **Learnability:** The ease with which the functioning of software can be learnt by the user
  - **Operability:** The ease with which the software can be operated by the user.
- **Efficiency:** The capability of software to perform with optimal resources under specified conditions is referred to as efficiency. The attributes of efficiency are listed below.
  - **Time behaviour:** Response time required by software to perform a function
  - **Resource behaviour:** Resources required by software to perform its functions.
- **Maintainability:** The capability of software to make specific modifications, which include correcting, improving, or adapting the software is referred to as maintainability. The attributes of maintainability are listed below.
  - **Testability:** Ability of software to be easily tested and validated
  - **Changeability:** Ability of software to be modified for fault removal and environmental change
  - **Analyzability:** Ability of software to allow detection of cause of faults and failures
  - **Stability:** Ability of software to withstand the risks occurring due to modifications.
- **Portability:** The capability of software to be transferred from one hardware or software environment to another. The attributes of portability are listed below.

NOTES

- **Installability:** Effort with which software is installed in a specified environment (both hardware and software environment)
- **Conformance:** Effort with which software complies with the standards that are related to portability
- **Replaceability:** Effort with which software is used in place of other software for a particular environment
- **Adaptability:** Ability of software to adapt to a new environment or new technology.



Fig. 14.2 *ISO 9126 Quality Factors*



Fig. 14.3 *ISO 9126 Quality Sub-attributes*

McCall's Quality Factors

There are several factors which are used as a measure to enhance quality of software. These factors are referred to as McCall's quality factors. As shown in Figure 14.4, these factors are characterized according to the following three important aspects.

- **Product operation:** Checks whether the software is able to perform the desired function efficiently. It comprises the following factors.

NOTES

- o **Correctness:** The effort with which software programs perform according to user requirements
- o **Reliability:** The effort with which the software is expected to perform its specified functions accurately
- o **Usability:** The effort with which software programs are understood and operated
- o **Integrity:** The degree to which unauthorized access to the software can be controlled
- o **Efficiency:** The amount of software code and computing resources required by software programs to carry out specified functions.



Fig. 14.4 *McCall's Quality Factors*

- **Product revision:** Checks whether the software requires effort to modify it. It comprises the following factors.
  - o **Maintainability:** The effort with which software can be easily maintained so that less time is taken to fix errors
  - o **Flexibility:** The effort with which a software program is modified
  - o **Testability:** The effort required to test a software program to ensure that it performs the required functions.
- **Product transition:** Checks whether the software is adaptable to the new environment. It comprises the following factors.
  - o **Portability:** The effort required to transfer a software program from one hardware/software environment to another
  - o **Reusability:** The degree to which a software program can be reused in other applications depending on the scope and functions performed by other software programs
  - o **Interoperability:** The effort with which a system integrates with another system.

NOTES

14.3 SOFTWARE QUALITY ASSURANCE GROUP

Software quality assurance (SQA) comprises various tasks that are responsible for ensuring quality. These tasks are assigned to software engineers and the SQA group. The common task of software engineers and the SQA group is to verify that the software is developed according to the user requirements and established standards.

SQA group comprises the *quality head*, *quality analyst*, and *quality control staff*. This group is concerned with keeping track of responsibilities of each team member and ensuring that tasks are being performed properly so that high-quality software is developed. The **quality head** is responsible to check that all the individuals of SQA group are performing the assigned tasks correctly. The **quality analyst** is responsible for establishing plans and procedures for evaluating software quality. Furthermore, the quality analyst reviews the software project and ensures that software quality is according to the established standards. **Quality control staff** is responsible for checking the functionality and performance of software by executing it and conducting different kinds of testing to detect errors.

Various functions performed by the SQA group are discussed here.

- **Preparation of SQA plan:** This plan is made in the initial stages of development (during process planning) and is reviewed by the management who is responsible for making decisions and policies in an organization as well as assigning and controlling tasks to be performed. SQA plan determines the tasks that are to be carried out by software engineers and the SQA group. In addition, it identifies the audits and reviews to be performed and standards and documents to be produced by the SQA group.
- **Participation in the software development process description:** A process is chosen by the software developers while developing the software. The SQA group assists the software development team by reviewing the process description and checking whether it is in accordance with the user requirements and established standards. The choice of process depends on the complexity of the software project.
- **Reviewal of activities in compliance with the defined software process:** The SQA group examines the process in order to record the changes made to it. In addition, it ensures that project documentation is updated to accommodate accurate records, to further ensure proper understanding during software maintenance. With incorrect records, the process cannot be evaluated accurately.
- **Verification of compliance with defined standards:** The SQA group assesses the process and compares it with established procedures and standards so that quality is incorporated in the software during each phase of software development.

Software Quality Assurance Plan

NOTES

Quality planning is a structured process for defining the procedures and methods, which are used to develop software. Quality planning starts in the early phases of software development. The SQA plan defines the activities for the SQA group. The objective of the SQA plan is to identify the quality assurance activities that are followed by the SQA group. Generally, the plan comprises documentation such as project plan, test plans and user documents. Figure 14.5 shows an SQA plan designed by IEEE.

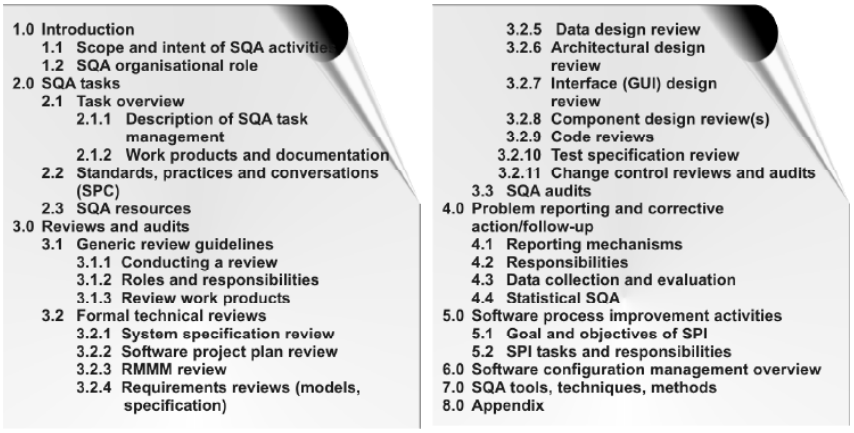


Fig. 14.5 *SQA Plan*

This plan comprises eight sections. The description of each section is discussed here.

- **Introduction:** Provides an overview of the entire SQA plan and SQA activities being performed in the software project. In addition, it specifies the role of organization in SQA.
- **Tasks:** Provides information about the SQA tasks performed in the SQA plan. It assigns SQA tasks to the software engineers and the SQA group. In addition, it describes SQA resources such as hardware, software, and tools to perform SQA tasks.
- **Reviews and audits:** Describes project reviews conducted by the SQA group. For performing reviews and audit, a set of guidelines is used. Furthermore, it provides information on formal technical reviews. Audits are used to assess SQA activities conducted in the project.
- **Problem reporting and corrective action/follow-up:** Describes problem-reporting mechanisms that occur as a result of reviews and provides corrective actions to improve them. Reporting mechanisms include the process of reporting problems (such as errors in software) to SQA group. In addition, it collects erroneous data and finds measures to correct errors.

NOTES

- **Software process improvement activities:** Describes the activities required for Software Process Improvement (SPI). It also outlines the objectives and tasks of SPI.
- **Software configuration management (SCM) overview:** Provides an overview of SCM plan. This plan provides information such as description of configuration management process and procedures to perform SCM.
- **SQA tools, techniques, and methods:** Describes the SQA tools, techniques, and methods that are used by the SQA group.
- **Appendix:** Provides additional information about SQA plan.

14.4 SOFTWARE QUALITY ASSURANCE ACTIVITIES

To ensure that the software under development is in accordance with established standards, SQA activities are followed. These activities are concerned with verification and validation of the software process being followed to accomplish the user requirements in the software. When SQA activities are complete, a report describing the results is presented to the management. If the report indicates that the software quality is not according to the user requirements then the management takes necessary actions to improve the quality of the software. Broadly, software quality assurance activities are classified into *process quality assurance activity* and *product quality assurance activity*.

Process Quality Assurance Activity

This activity ensures that the processes that are defined for the project are followed. In case the defined processes are not followed according to standards, suggestions for those situations are provided to the management. A separate report is sent to both the management and the SQA group. The SQA group examines associated problems occurring due to delay in schedule or increase in budget and reports them to the management. The process quality is evaluated through software process audits. These audits focus on checking the implementation of the defined processes in software. Software process audits are conducted by quality analysts from time to time. Various tasks involved in process quality assurance activity are listed below.

- Ensuring development of project plans and their review and approval
- Conducting software configuration reviews
- Establishing software quality plans, standards, and procedures for the project
- Establishing periodic reviews with the user

- Submitting a profile to the management about the adherence of defined process throughout the software development process, by showing regular improvement in the development process
- Ensuring regular submission of metrics and other required information to the software engineering process group (SEPG).

Product Quality Assurance Activity

This activity identifies the processes in order to check whether they are carried out properly to develop a product that conforms to the user requirements. Product quality assurance activity concentrates on detection and elimination of errors as early as possible during software development. As a result, there is reduction in costs incurred during software maintenance and testing. Product quality assurance comprises methods to conduct reviews and testing of the software. Reviews evaluate the software before delivering it to the user. After software reviews are complete, the quality analyst performs a final review. Final review is essential to verify that all planned reviews or reported problems are followed. In addition to reviews, software testing is performed to check errors in software and to verify whether the errors are occurring due to non-conformance to the user requirements.

Auditing in Software Quality Assurance Activities

SQA activities follow several techniques to find errors in the software. Audit is a fundamental technique used for both process quality assurance and product quality assurance. It examines a process or product in detail and compares it with the established standards. Audit is performed to examine the conformance of a development process to procedures and standards. In addition, it examines the records according to the defined process to determine whether the procedures are followed correctly. The auditor performs audit to examine whether the records, procedures, and management policies are executed properly. An auditor can be internal—one of the employees of the organization—as well as external, which is selected by the organization and SEPG.

14.5 SOFTWARE REVIEWS

Software reviews are systematic evaluation of the software in order to detect errors. These reviews are conducted in various phases of the software development process such as analysis, design, and coding. Note that software reviews are conducted for the product, software, or documentation. The objective of software reviews is to ensure that the quality of software is maintained in each phase of the software development. The process commonly followed for conducting software reviews is shown in Figure 14.6.

NOTES

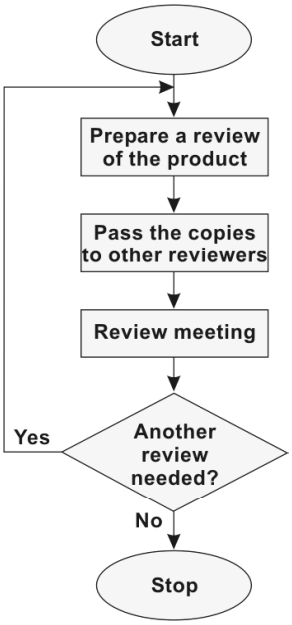


Fig. 14.6 *Review Process*

A review team performs software reviews (see Figure 14.7). The team members that constitute a review team are listed below.

- **Author or producer:** Develops the product and is responsible for making corrections in it after the review is over. The author raises issues concerning the product during review meeting.
- **Moderator or review leader:** The review leader performs the following activities.
  - o Ensures that review procedures are performed throughout the review meeting
  - o Ensures that the review team is performing their specified responsibilities
  - o Verifies the product for review
  - o Assembles an effective team and keeps review meetings on schedule
  - o Organizes a schedule of the product to be reviewed.
- **Recorder:** Records important issues of the product, which arise during a review meeting. A document is prepared containing the results of the review meeting. This document includes type and source of errors.
- **Reviewer or inspector:** Analyzes the product and prepares notes before the review meeting begins. There can be more than one reviewer in the team depending upon the size of the product to be reviewed.



Fig. 14.7 *Review Team*

Different types of reviews are conducted as a part of the review process. The software review can be conducted in both formal and informal ways. A review meeting can be conducted in a cafeteria with the purpose of solving technical problems. Similarly, a meeting held where some agenda is discussed formally is also a form of software review. Generally, software reviews are of two types, namely, *walkthroughs* and *formal technical reviews*.

Walkthroughs

This review is an informal review, which is used to evaluate the product. No preparation is made in advance and due to this, walkthroughs are termed as an unplanned activity. In a walkthrough, the author describes the product to the peers (group of individuals other than the author) and gets feedback in an informal way (see Table 14.1). Note that the people involved in this review do not have any specified responsibility.

Table 14.1 *Tasks in a Walkthrough*

Tasks	*Evaluators*
Pass the product to the peer for a walkthrough.	Author
Suggestions for improving the software are presented to the author.	Peer members
Implement necessary rework on the product according to the suggestions of peer members.	Author

As walkthroughs are informal, they are not considered efficient. This is because walkthroughs leave several errors unnoticed. In addition, no record of errors is maintained. It results in difficulties in verifying the correctness of the product after walkthroughs are 'over'.

NOTES

Formal Technical Review (FTR)

A formal technical review (FTR) is a formal review that is performed by a review team. It is performed in any phase of software development for any work product, which may include requirements specification, design document, code, and test plan. Each FTR is conducted as a meeting and is considered successful only if it is properly planned, controlled, and attended. The objectives of FTR are listed below.

- To detect errors in functioning of software and errors occurring due to incorrect logic in software code
- To check whether the product being reviewed accomplishes user requirements
- To ensure that a product is developed using established standards.

Review Guidelines

To conduct a formal technical review, there are some review guidelines. These guidelines are established before a review meeting begins. The commonly followed review guidelines are listed below.

- **Review the product:** The focus of formal review should be to detect errors in the product instead of pointing out mistakes (if any) committed by a team member. The aim should be to conduct the review in harmony among all the team members.
- **Set the agenda:** Formal reviews should keep a track of the schedule. The moderator is responsible for maintaining this schedule. For this purpose, he ensures that each review team member is performing the assigned task properly.
- **Keep track of discussion:** During the review meeting, various issues arise and it is possible that each review team member has a different view on an issue. Such issues should be recorded for further discussion.
- **Advance preparation:** Reviewers should make an advance preparation for the product to be reviewed. For this purpose, the reviewers should note the issues that can arise during the review meeting. Then, it is easy for them to discuss the issues during the review meeting.
- **Indicate problems in the product:** The objective of a review meeting should be only to indicate the problems or errors. In case there are no proper suggestions for the problems, a review meeting should be conducted again.
- **Categorize the error:** The errors detected in the software should be classified according to the following categories.
  - **Critical errors:** Refer to errors that bring the execution of the entire software to a halt. Thus, crucial errors need to be 'fixed' before the software is delivered.

NOTES

- **Major errors:** Refer to errors that affect the functionality of programs during their execution. Like critical errors, major errors need to be fixed before the software is delivered
- **Besides errors:** Refer to errors that do not affect the usability of the software
- **No errors:** Indicates that there are no errors in the software.

- **Prepare notes:** The recorder who is one of the reviewers should keep a record of the issues in order to set priorities for other reviewers as the information is recorded.
- **Specify the number of people:** There should be a limited number of individuals in the meeting that should be specified before the meeting begins.
- **Develop a checklist:** A checklist should be maintained at the end of the meeting. The checklist helps the reviewer to focus on important issues that are discussed at the meeting. Generally, a checklist should be prepared for analysis, design, and code documents.

Review Meeting

A review meeting is conducted to review the product in order to validate its quality. Review team members examine the product to identify errors in it. As shown in Figure 14.8, a successful review consists of a number of stages, which are described here.

- **Planning:** In this stage, the author studies the product that requires a review and informs the moderator about it. The moderator verifies the product that is to be examined. The verification is essential to determine whether the product requires a review. After verification, the moderator assigns tasks to the review team members. The objective of planning is to ensure that the review process is followed in an efficient manner. In addition, it ensures that a proper schedule is made to conduct an effective review.
- **Overview:** In this stage, the product is analyzed in order to detect errors. For this purpose, knowledge of the product is essential. In case reviewers do not have proper knowledge of the product, the author explains the functionality and the techniques used in the product.
- **Preparation:** In this stage, each review team member examines the product individually to detect and record problems (such as errors and defects) in the product. There are several types of problems that are identified during the preparation stage. Generally, the problems considered at this stage are listed below.
  - **Clarity:** User requirements are not understood properly
  - **Completeness:** Details of user requirements are incomplete
  - **Consistency:** Names of data structures and functions are used illogically

NOTES

- **Functionality:** Functions, inputs, and outputs are not specified properly
- **Feasibility:** Constraints such as time, resources, and techniques are not specified correctly.

- **Meeting:** In this stage, the moderator reviews the agenda and issues related to the product. The problems described in the overview stage are discussed among review team members. Then, the recorder records the problems in a defect list, which is used to detect and correct the errors later by the author. The defect list is divided into the following categories.
  - **Accept the product:** In this category, the author accepts the product without the need for any further verification. This is because there are no such problems that halt the execution of the product.
  - **Conditionally accept the product:** In this category, the author accepts the product, which requires verification. If there are problems in the product, the next stage (rework stage) is followed.
  - **Re-examine the product:** In this category, the author re-examines the product to understand the problems in it. After rework, the product is sent to the moderator again to verify that problems are eliminated.
- **Rework:** In this stage, the author revises the problems that are identified during the review meeting. He determines the problems in the product and their causes with the help of a defect list. Then, he resolves the problems in the product and brings it back to the moderator for the follow-up stage.
- **Follow-up:** In this stage, the product is verified after the author has performed rework on it. This is due to the fact that he may have introduced new errors in the product during rework. In case there are still some errors in the product, a review meeting is conducted again.

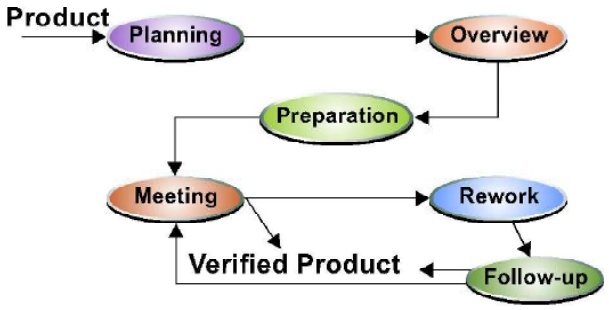


Fig. 14.8 *Stages of Review Meeting*

While the review is being conducted, the recorder records all the issues raised during the review meeting. After he completes the task, the reviewer summarizes the review issues. These issues are recorded in two kinds of documents, which are listed below.

- **Review Issue list:** This document is concerned with the identification of problems in the product. It also acts as a checklist that informs the author about the corrections made in the product.
- **Review summary report:** This document focuses on information such as the phase of software development that is reviewed, the review team member who reviewed it, and conclusions of the review meeting. Generally, a review summary report comprises a single page and is advantageous for future reference by the software development team.

Cost Impact of Software Errors

Formal technical reviews are used for detecting errors during the development process of the software. This implies that reviews detect errors and thereby reduce the cost of software and increase its efficiency. Thus, the formal technical reviews provide cost-effective software.

For example, Figure 14.9 (a) shows the errors present in software before conducting a formal technical review. When FTR is conducted, there is reduction in errors, as shown in Figure 14.9 (b). Here, approximately half of the errors are fixed and some part of software still contains errors in it. These errors can occur when some of the errors are fixed and due to it, new errors arise in the software. It is observed that FTR conducted in early stages of the software development process reduces the time and cost incurred in detecting errors in software.

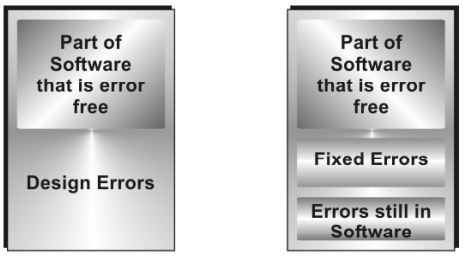


Fig. 14.9 *Detection of Errors*

14.6 EVALUATION OF QUALITY

Various kinds of approaches are used to improve quality of the software and overall process of the organization. The commonly used software quality approaches are *Six Sigma* and *ISO standards*.

Six Sigma

Today, Six Sigma is regarded as the most widely used strategy for evaluating quality. Six Sigma refers to six standard deviations. The standard deviation describes

NOTES

the distribution of the process or procedure used in software development. It is a statistical strategy to ensure quality assurance of processes in the organization. Thus, it can be stated that Six Sigma focuses on identification and prevention of errors in the software development process. In addition, it is used to eliminate errors in the software by evaluating the performance of the software process. The six standard deviations used in Six Sigma strategy are listed below.

- Good business strategy that balances cost, quality, features, and constraints as well as matches business priorities
- Decisions made are tied to bottom line of the organization
- Exercise care to use correct measurements in each situation
- Consider measuring output for a longer period
- Limited scope with adequate importance and reasonable time
- Clear quantitative measures to define success.

The objective of Six Sigma is to reduce variance (an activity that varies from a norm or a standard) and improve the processes in an organization. In addition, it minimizes the cost of poor quality. These objectives are directly associated with the objectives of an organization. Six Sigma monitors day-to-day activities of an organization, in order to make optimal use of resources. According to Six Sigma, processes in an organization can be improved by applying the following approaches.

- **DMAIC:** It stands for define, measure, analyze, improve, and control. This approach checks whether a process is performing correctly. It improves the system by improving the process that is not able to meet user requirements. Various attributes and functions present in this approach are listed in Table 14.2.

Table 14.2 *DMAIC Methodology*

Attributes	***Functions***
Define	Identify goals of the project and the user requirements.
Measure	Quantify the process to determine the current performance of the software.
Analyze	Examine and determine the cause of the error.
Improve	Improve the process by eliminating the errors.
Control	Determine future process performance for prevention of errors.

- **DMDAV:** It stands for define, measure, design, analyze, and verify. This approach is used while a software process is being developed in the organization. Various attributes and functions present in this approach are listed in Table 14.3.

NOTES

Table 14.3 DMDAV Methodology

Attributes	*Functions*
Define	Specify goals of the project and the user requirements.
Measure	Evaluate and determine the user requirements.
Design	Design the process to avoid the root cause of defect.
Analyze	Study process options to meet the user requirements.
Verify	Ascertain the design performance so that the user requirements are accomplished.

Note: In order to achieve Six Sigma, the errors should not be more than 3.4 per million occurrences.

ISO 9000 Quality Standards

The ISO 9000 standards describe the elements in quality assurance, which outline the requirements of good quality product in an organization. The elements comprise organizational structure, procedures, and resources that are needed to implement quality assurance in the software. These standards also provide auditing tools to make sure that they are properly implemented according to the standards and meet the user requirements. After implementing these standards, it is mandatory to audit the organization to evaluate its effectiveness. After a successful audit, the organization receives a registration certificate, which identifies the quality of the organization as being in compliance with ISO 9000 standards. ISO 9000 series includes three models, which are listed below.

- **ISO 9001:** This quality assurance model applies to organizations that design, develop, install, and service products. It discusses how to meet customer needs effectively.
- **ISO 9002:** This quality assurance model applies to organizations that produce, install, and service products. This model is nearly identical to 9001, except that it does not incorporate design and development.
- **ISO 9003:** This quality assurance model applies to organizations whose processes are almost exclusive to inspection and testing of final products. This model is currently limited to an inspection function.

One of the quality standards that is commonly used in practice is ISO 9001:2000. It is a quality management standard that is established for improving the overall quality of the process and product in an organization. It can be achieved by using appropriate management policies and practices. The ISO 9001:2000 is based on several management principles relating to different requirements. These principles are discussed here.

- **Management responsibility:** Describes a policy and regular reviews of management tasks and activities
- **Sales:** Provides information, which is helpful in understanding customer needs

NOTES

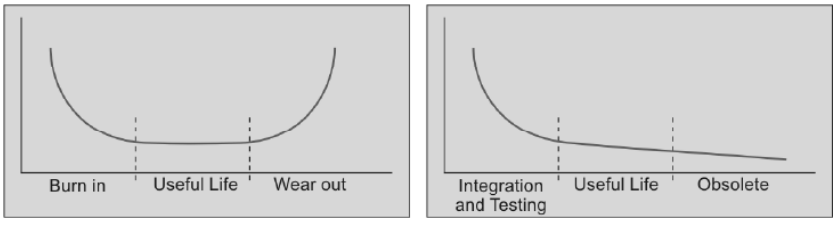
- **Document control:** Provides information on manuals and documentation prepared for an organization
- **Purchasing:** Provides information to customers on how to make sure that they are purchasing the required products
- **Identification and traceability:** Provides information on how to identify products and services (particularly certified items)
- **Inspection and test:** Provides information on how to test and inspect the products.
- **Corrective and preventive action:** Provides information on how to detect faults in the products and ways to manage them
- **Quality system:** Consists of a quality manual, which shows whether the manual applies to the management. It also examines the procedures and processes of the product.

14.7 SOFTWARE RELIABILITY

Software reliability is one of the quality factors that focus on the following tasks.

- Description of requirements for software, fault/ failure detection, and recovery of software in case of failure
- Review of software development process and products for software error prevention
- Description of processes for measuring and analyzing errors and description of reliability and maintainability factors for the software.

IEEE defines software reliability as '*the process of optimizing the reliability of software through a program that emphasizes software error prevention, fault detection and removal and the use of measurements to maximize reliability in light of project constraints, such as resources, schedule, and performance.*' Reliability constitutes two major activities, namely, *error prevention* and *fault detection and removal*.



(a) Hardware Failure Rate (b) Software Failure Rate

Fig. 14.10 *Hardware and Software Failure Rate*

A failure can be hardware failure or software failure. As shown in Figure 14.10 (a), hardware has three phases, namely, *burn in*, *useful life*, and *wear*

out. In the burn in phase, the failure rate is high and decreases with time for the next phase, which is useful life. As compared to the burn in phase, the useful life phase is constant. The failure rate increases in the wear out phase. This is because the hardware components wear out with time. As shown in Figure 14.10(b), the rate of software failure as compared to Figure 14.10(a) is less and software failure occurs largely in the integration and testing phase. As the software is tested, errors are detected and removed. This results in decreasing the failure rate for the next phase of the software known as useful life. The software does not wear out, but becomes obsolete due to factors such as changes in the technology or the version, increase in complexity, and so on.

Three approaches are used to improve the reliability of software. These approaches are listed below.

- **Fault avoidance:** The design and implementation phase of the software development uses the process that minimizes the probability of faults before the software is delivered to the user.
- **Fault detection and removal:** Verification and validation techniques are used to detect and remove faults. In addition, testing and debugging can also remove faults.
- **Fault tolerance:** The designed software manages the faults in such a way that software failure does not occur. There are three aspects of fault tolerance.
  - **Damage assessment:** This detects the parts of software affected due to occurrence of faults.
  - **Fault recovery:** This restores the software to the last known safe state. Safe state can be defined as the state where the software functions as desired.
  - **Fault repair:** This involves modifying the software in such a way that faults do not recur.

Software Reliability Models

A software reliability model is examined as a mathematical analysis model for the purpose of measuring and assessing software quality and reliability quantitatively. As shown in Figure 14.11, detection and removal of errors incur huge costs. When the probability of software failure occurrence decreases, software reliability increases. A model used to describe software reliability is known as **Software Reliability Growth Model** (**SRGM**). The software reliability model evaluates the level of software quality before the software is delivered to the user. The objectives of software reliability models are listed below.

- To evaluate the software quantitatively
- To provide development status, test status, and schedule status
- To monitor reliability performance and changes in reliability performance of the software

NOTES

- To evaluate the maintenance cost for faults that are not detected during the testing phase.

The software reliability models can be divided into two classes. The first class deals with the designing and coding phases of software development by analyzing the reliability factors of the product. The second class deals with the testing phase. It describes the software failure-occurrence phenomenon or software fault-detection phenomenon by applying statistics theories that estimate software reliability. The software reliability models are broadly classified into two categories, namely, *dynamic models* and *static models*.

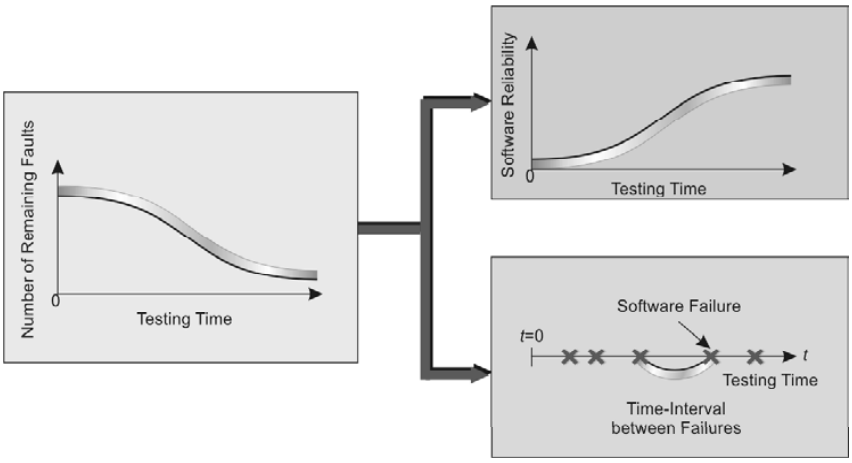


Fig. 14.11 *Software Reliability*

- **Dynamic models:** These models follow the changes of software during the testing phase. In addition, these models are based on the assumption that errors can be corrected as soon as they are detected. In other words, it is assumed that the time consumed in fixing errors is negligible.
- **Static models:** These models measure the quality of the software before debugging and testing.

Assessing Reliability and Availability

The measure of reliability is defined as the probability of software to operate without failures under given conditions. The ability to measure or predict reliability in software engineering focuses on three general purposes.

- Understanding and making trade-offs between reliability and other software characteristics such as performance, cost, and schedule
- Tracking progress when testing is complete
- Determining the effect of using new methods to develop software.

A common measure of how long the system can operate between failure of critical part is 'mean time between failure' (MTBF). In other words, MTBF is

NOTES

defined as the average of times between successive failures. It is a statistical measure rather than a direct measure. A measure similar to MTBF is 'mean time to repair' (MTTR), which is the average amount of time taken to repair the machine after a failure occurs. It can be combined with 'mean time to failure' (MTTF), which describes how long the software can be used to calculate MTBF.

$$MTBF = MTTF + MTTR.$$

In addition to assessment of reliability, software availability is also calculated. Availability of software is defined as the probability of software to operate and deliver the desired request. Availability is an indirect measure to determine maintainability. This is because it determines the probability that software is working in accordance with the user requirements at a specified time. It is calculated from probabilistic measures MTBF and MTTR. In the mathematical notation, availability is defined as follows:

$$Availability = (1 - MTTR / (MTTR + MTBF))* 100\%.$$

Software Safety

Software safety is a software quality assurance activity that follows a systematic approach to identify, analyze, record, and control software hazards to ensure that software operates in an intended manner. A hazard is defined as a set of conditions that can lead to an accident in a given set of conditions. These conditions are beyond the control of the software developer, that is, the software developer is unable to predict the conditions that can cause software to stop its functioning.

A software hazard occurs when a wrong input is given. Some hazards are avoidable and can be eliminated by changing the design of the system while others cannot be avoided and must be handled by the software. A technique known as fault-free analysis is used to prevent hazards in the developed software. In this analysis a detailed study is carried out to detect conditions that cause hazards. Once the hazards are analyzed, the requirements for the software are specified.

It is important to note the difference between software safety and software reliability. Software reliability uses statistical and mathematical methods to identify software failure and it is not necessary that this occurrence of failure leads to a hazardous situation. Software safety identifies the conditions in which these failures lead to a hazardous situation.

Check Your Progress

1. Define software quality assurance.
2. What is quality costs?
3. What are software reviews?
4. What is software reliability model?
5. Define formal technical review.